

EVALUACIÓN SUMATIVA I · FASE 1

Aplicación Django del lado del servidor · EcoEnergy

ÁREA ACADÉMICA	Informática y Telecomunicaciones	CARRERA	Analista Programador
ASIGNATURA	Programación Back End	CÓDIGO	TI3041
SEDE	La Serena	DOCENTE	Javier Ahumada Cortés
UNIDAD	Unidad 1	INSTRUMENTO	Escala de Apreciación
EVALUACIÓN	Sumativa I · EcoEnergy	PUNTAJE FASE 1	60 puntos
PONDERACIÓN	60% de la evaluación	EXIGENCIA FINAL	60% para nota 4,0
PUBLICACIÓN FASE 1	_____	CIERRE FASE 1	_____
EVALUACIÓN OFICIAL	_____	HORARIO	_____

Alcance de esta entrega

Este documento contiene la Fase 1, equivalente a 60 puntos y al 60% de la Evaluación Sumativa I. La evaluación constituye un solo instrumento de 100 puntos. El día de la evaluación oficial, todo el curso recibirá el instrumento completo, incluida la Fase 2 de 40 puntos.

1. Aprendizaje esperado y criterios de evaluación

Aprendizaje esperado 1.1: utiliza tecnologías del lado del servidor para resolver problemáticas planteadas, de acuerdo con la sintaxis del lenguaje.

- 1.1.1 Identifica variables y operaciones del lenguaje, según requerimiento.
- 1.1.2 Codifica instrucciones, estructuras y operadores, según requerimientos.
- 1.1.3 Codifica instrucciones utilizando paquetes externos, según requerimientos.
- 1.1.4 Implementa una aplicación sencilla en Django, según requerimientos.

2. Condiciones de publicación y asistencia

1. La publicación de este archivo en el Ambiente de Aprendizaje (AMBIENTE DE APRENDIZAJE) constituye la entrega oficial del enunciado de Fase 1.
2. La inasistencia a la clase de presentación no modifica por sí sola el plazo. El estudiante debe revisar las instrucciones publicadas.
3. Las ausencias justificadas y adecuaciones de plazo se resolverán conforme a la normativa institucional y deberán quedar autorizadas por escrito.
4. El día de la evaluación oficial todos los estudiantes recibirán el instrumento completo. Esta entrega no reabre el cierre de Fase 1, salvo autorización escrita.
5. Toda aclaración que cambie alcance, criterios o entrega será publicada para el curso; una indicación verbal no reemplaza la información escrita.

3. Trabajo y entrega de la Fase 1

La Fase 1 es individual y asistida. Cada estudiante debe construir, probar, documentar y comprender su propia solución.

- Permitido: clases, apuntes, documentación oficial, pruebas iterativas y orientación conceptual.
- Permitido: utilizar IA para apoyar análisis, HTML, Bootstrap o revisión de código, siempre que IA.md registre herramienta, prompts, partes utilizadas, cambios propios y pruebas realizadas.
- No permitido: compartir archivos, copiar un repositorio completo o presentar código que el estudiante no pueda explicar.
- La versión evaluada corresponde al hash entregado dentro del plazo. Los commits posteriores no modifican el puntaje de Fase 1.
- La entrega es válida cuando el AMBIENTE DE APRENDIZAJE contiene la URL del repositorio y el hash exacto solicitado.

Uso responsable de IA

La IA puede proponer una estructura HTML o estilos Bootstrap, pero el estudiante debe seleccionar, adaptar, integrar y probar la respuesta. Copiar una salida sin comprenderla o sin verificarla no constituye evidencia de logro.

4. Contexto del caso EcoEnergy

EcoEnergy ofrece a pequeñas y medianas empresas una aplicación para consultar zonas de consumo energético y los dispositivos instalados en ellas. Esta primera versión utiliza archivos JSON como fuente de datos. Django debe cargar las colecciones, resolver sus relaciones mediante estructuras Python y presentar los resultados mediante Templates Bootstrap.

La aplicación debe permitir listar zonas y consultar su detalle. El detalle informa dispositivos, categorías, consumo total y estado respecto del límite permitido. La solución debe continuar funcionando cuando aumente la cantidad de registros válidos o una zona no posea dispositivos.

Alcance obligatorio

No se solicitan Models, migraciones, ORM, CRUD, formularios, autenticación, permisos, soft delete ni múltiples organizaciones. Agregarlos no entrega puntaje adicional.

4.1 Modelo UML entregado



Las relaciones deben implementarse mediante identificadores en archivos JSON.

Figura 1. Modelo de datos requerido para la Fase 1.

4.2 Archivos JSON obligatorios

Archivo	Claves obligatorias	Mínimo inicial
zonas.json	id, nombre, limite_kwh	3 registros
categorias.json	id, nombre, descripcion	3 registros
dispositivos.json	id, nombre, consumo_kwh, zona_id, categoria_id	8 registros

Todos los identificadores deben ser únicos dentro de su archivo. Cada zona_id y categoria_id debe referenciar un registro existente. Las claves y tipos deben mantenerse uniformes. Las pruebas docentes conservarán una estructura JSON válida.

5. Historia de usuario y criterios de aceptación

Historia de usuario

Como encargado del monitoreo energético de EcoEnergy, quiero consultar las zonas de consumo y revisar el detalle de sus dispositivos, para identificar su consumo total y reconocer oportunamente estados normales, alertas o ausencia de información, mediante una interfaz clara que continúe funcionando cuando cambie la cantidad de datos registrados.

Código	Criterio de aceptación verificable
CA-01	El listado muestra todas las zonas registradas en zonas.json.
CA-02	Cada zona muestra nombre, límite, cantidad de dispositivos y acceso al detalle.
CA-03	El detalle muestra dispositivos, categoría, consumo, métricas y estado de la zona.
CA-04	Las cantidades, sumas y estados se calculan dinámicamente; no quedan escritos manualmente en el HTML.
CA-05	ALERTA cuando consumo_total > limite_kwh y NORMAL cuando consumo_total <= limite_kwh.
CA-06	Al agregar registros válidos al JSON, la aplicación los incorpora sin modificar una View o Template por cada elemento.
CA-07	Una zona sin dispositivos mantiene la aplicación operativa y muestra un mensaje comprensible.
CA-08	Un identificador de zona inexistente responde de forma controlada mediante 404.
CA-09	La interfaz conserva su estructura al aumentar zonas o dispositivos; la navegación y los controles permanecen accesibles.
CA-10	Las tablas extensas permiten desplazamiento dentro de un contenedor adaptable, sin desbordar la página completa.
CA-11	Header, navegación, títulos, tablas, tarjetas, botones y mensajes mantienen una jerarquía visual coherente.
CA-12	Los estados utilizan texto y apoyo visual; el color no es el único medio para comunicar significado.
CA-13	El proyecto puede instalarse desde el repositorio, ejecutar Django y superar python manage.py check.

6. Pruebas mínimas de funcionamiento

Durante la revisión, el docente podrá modificar temporalmente el archivo JSON, conservando su estructura válida. La aplicación deberá procesar los cambios dinámicamente, sin exigir modificaciones manuales en las Views o Templates por cada registro agregado.

Escenario	Acción de comprobación	Resultado esperado
1. Nuevos registros	Agregar dos dispositivos válidos.	Aparecen automáticamente y se actualizan cantidades, consumo y estado.
2. Mayor volumen	Duplicar temporalmente varios registros válidos.	La interfaz conserva estructura, navegación y acceso al contenido.
3. Colección vacía	Dejar una zona sin dispositivos.	Se muestra un mensaje claro y la aplicación continúa funcionando.
4. Identificador inexistente	Solicitar una zona que no existe.	Respuesta 404 controlada, sin exponer una falla técnica al usuario.
5. Estados	Usar datos que produzcan NORMAL y ALERTA.	Texto y señal visual corresponden correctamente a la regla definida.

No se evaluará la recuperación frente a errores de sintaxis del archivo JSON —por ejemplo, una coma o llave eliminada— porque esa condición no forma parte del alcance de esta fase.

7. Bocetos de interfaz

Carácter referencial

Los bocetos definen información mínima y jerarquía general. No exigen copiar colores, medidas o distribución exacta. El estudiante puede proponer otro diseño Bootstrap si conserva todos los datos, estados y acciones solicitados.

Boceto 1. Listado de zonas

Referencia de información mínima. No se exige una réplica exacta de distribución, colores o tipografía.

ECOENERGY Inicio Zonas		
ZONAS DE CONSUMO		
ZONA 1 Límite: ____ kWh Dispositivos: ____ [Ver detalle]	ZONA 2 Límite: ____ kWh Dispositivos: ____ [Ver detalle]	ZONA 3 Límite: ____ kWh Dispositivos: ____ [Ver detalle]
Estado alternativo: "No hay zonas disponibles".		

Boceto 2. Detalle de una zona

La interfaz debe comunicar información y estados; la lógica para obtenerlos debe resolverse en Python.

ECOENERGY Zonas Volver al listado															
DETALLE DE ZONA: _____															
Límite ____ kWh	Consumo total ____ kWh	Dispositivos ____	Estado NORMAL / ALERTA												
<table><thead><tr><th>Dispositivo</th><th>Categoría</th><th>Consumo kWh</th></tr></thead><tbody><tr><td>_____</td><td>_____</td><td>_____</td></tr><tr><td>_____</td><td>_____</td><td>_____</td></tr><tr><td>_____</td><td>_____</td><td>_____</td></tr></tbody></table>				Dispositivo	Categoría	Consumo kWh	_____	_____	_____	_____	_____	_____	_____	_____	_____
Dispositivo	Categoría	Consumo kWh													
_____	_____	_____													
_____	_____	_____													
_____	_____	_____													
Estado alternativo: "Esta zona no tiene dispositivos". Un id de zona inexistente debe responder con 404.															

Observación sobre el diseño

El diseño será evaluado por su funcionalidad y capacidad para comunicar información, no por apreciaciones subjetivas sobre creatividad o belleza. Los colores deberán poseer un significado comprensible; las alertas deberán distinguir éxito, advertencia y error; las tablas deberán presentar encabezados claros; y el header, la navegación y la distribución del contenido deberán corresponder a la estructura de un sistema web. Los estados no podrán comunicarse exclusivamente mediante colores: deberán incorporar texto, etiquetas o íconos.

Usabilidad y comportamiento de la interfaz

La interfaz deberá conservar una estructura comprensible cuando aumente la cantidad de información. El contenido no deberá superponerse, salir del área visible ni impedir el acceso a botones, navegación o resultados.

Las tablas extensas deberán mantenerse dentro de un contenedor adaptable y permitir desplazamiento cuando corresponda. Los mensajes de error, advertencia, ausencia de datos y operación exitosa deberán utilizar textos claros y componentes visuales coherentes. La evaluación considerará claridad, accesibilidad y estabilidad de la interfaz, no preferencias estéticas personales.

La revisión se realizará en el navegador del laboratorio con zoom de 100%. Se aceptará desplazamiento horizontal dentro de tablas extensas, pero no el desbordamiento general de la página ni la pérdida de acceso a la navegación o a los controles.

8. Entregables obligatorios

- Repositorio individual accesible para el docente.
- Código fuente Django y los tres archivos JSON solicitados.
- ANALISIS.md: relaciones, multiplicidades, claves de conexión y matriz Criterio de aceptación | Archivo/Componente | Prueba.
- IA.md: herramienta, prompts, respuesta utilizada, cambios propios y verificación. Si no se utilizó IA, declarar "No se utilizó IA".
- README.md: requisitos, instalación, ejecución, rutas funcionales y pruebas.
- requirements.txt y .gitignore actualizados.
- Al menos cuatro commits progresivos y comprensibles que representen etapas reales del desarrollo.
- En el AMBIENTE DE APRENDIZAJE: URL del repositorio y hash exacto del commit "Entrega fase 1 evaluacion unidad 1".

9. Reglas de entrega y verificación

- Un repositorio existente sin URL y hash entregados en el AMBIENTE DE APRENDIZAJE no constituye una entrega formal.
- Si el push falla al cierre, solo se considerará el commit local cuando el incidente se informe antes del vencimiento y el docente registre el hash en ese momento.
- Durante la revisión, el docente podrá agregar registros válidos, aumentar dispositivos o dejar una colección sin elementos para verificar comportamiento dinámico y estabilidad.
- Las funcionalidades adicionales no compensan requisitos obligatorios ausentes.
- Los archivos .env, credenciales, claves o información personal no deben incluirse en el repositorio.
- El proyecto debe ejecutar desde el repositorio siguiendo el README y superar python manage.py check.
- Mensajes como "cambios", "avance", "final" o "ahora sí" no demuestran una progresión comprensible del trabajo.

10. Lista de comprobación

- ☐ El proyecto ejecuta sin errores con python manage.py check.
- ☐ Los tres JSON son válidos, cumplen los mínimos y poseen relaciones consistentes.
- ☐ Las rutas /zonas/ y /zonas/<id>/ funcionan y son accesibles desde la navegación.
- ☐ NORMAL, ALERTA, estado vacío y 404 fueron probados.
- ☐ Al agregar registros válidos, la información, cantidades y resultados se actualizan sin modificar el código por cada elemento.
- ☐ Con más datos, la interfaz mantiene navegación, controles y contenido accesibles.
- ☐ Los Templates heredan de base.html y utilizan Bootstrap con significado visual coherente.
- ☐ requirements.txt, .gitignore, README.md, ANALISIS.md e IA.md están incluidos.
- ☐ El código presenta nombres comprensibles e indentación consistente de cuatro espacios.
- ☐ El repositorio no contiene .env, credenciales ni archivos personales.
- ☐ Existen al menos cuatro commits progresivos y el hash final fue entregado en el AMBIENTE DE APRENDIZAJE.

11. ESCALA DE APRECIACIÓN · FASE 1

Puntaje máximo: 60 puntos · Marque un solo nivel por criterio

Criterio	Destacado	Habilitado	En desarrollo	No logrado
F1-1 · 1.1.1 Análisis y representación de datos Máx. 10 pts	Interpreta el caso y representa correctamente zonas, categorías y dispositivos. Identificadores, tipos y relaciones son coherentes y verificables. 10 pts	Representa los datos y relaciones principales; presenta una omisión menor que no impide utilizar la información. 8 pts	Representa parcialmente los datos o presenta inconsistencias que afectan algunos resultados. 5 pts	Los datos no están representados, no se relacionan o no existe evidencia verificable. 0 pts
F1-2 · 1.1.2 Procesamiento y robustez del JSON Máx. 12 pts	Carga, recorre, filtra y relaciona dinámicamente los datos. Procesa nuevos registros válidos, colecciones vacías, zonas sin dispositivos e identificadores inexistentes sin detener la aplicación. 12 pts	Procesa nuevos registros y operaciones principales; presenta un error menor en un caso excepcional, pero mantiene el funcionamiento general. 9 pts	Funciona solo con parte de los datos o presenta errores al aumentar, eliminar o relacionar registros. 6 pts	Depende de datos escritos manualmente, se detiene al modificar el JSON o no permite comprobar el procesamiento. 0 pts
F1-3 · 1.1.3 Paquetes externos Máx. 6 pts	Instala, configura y utiliza una dependencia pertinente; la registra en requirements.txt y justifica claramente su aporte. 6 pts	La dependencia funciona y está registrada; presenta una omisión menor en configuración o justificación. 4 pts	Existe un intento verificable, pero la dependencia se usa parcialmente o la instalación no es completamente reproducible. 2 pts	No utiliza una dependencia externa pertinente o no existe evidencia verificable. 0 pts
F1-4 · 1.1.4 Aplicación Django e integración MVT Máx. 20 pts	Rutas, Views, contexto, listado, detalle, navegación, cálculos, estados, casos vacíos y 404 funcionan correctamente y mantienen responsabilidades MVT. 20 pts	Cumple los requerimientos principales; presenta un error menor o un requisito secundario incompleto. 15 pts	La integración funciona parcialmente; una ruta o funcionalidad relevante presenta errores o requiere intervención. 10 pts	La aplicación no ejecuta o no permite verificar la integración entre rutas, Views y Templates. 0 pts
F1-5 · 1.1.4 Usabilidad y significado visual Máx. 6 pts	La interfaz Bootstrap mantiene estructura con más datos; no oculta contenido ni controles; tablas extensas permiten desplazamiento y estados usan texto más apoyo visual comprensible. 6 pts	La interfaz es funcional y mantiene accesible la información principal, con desajustes menores que no impiden utilizarla. 4 pts	Presenta desbordamientos, desalineaciones o dificultades de navegación que afectan parcialmente el uso. 2 pts	La interfaz se descuadra, oculta información o no permite interpretar y utilizar la funcionalidad. 0 pts
F1-6 · Transversal Git, entrega y calidad del código Máx. 6 pts	Repositorio verificable con al menos 4 commits progresivos y mensajes claros; gitignore, documentación, URL y hash correctos. Código legible e indentación consistente de 4 espacios. 6 pts	Entrega verificable con al menos 3 commits comprensibles; existen detalles menores en progresión, documentación o formato del código. 4 pts	Presenta 1 o 2 commits, mensajes genéricos o problemas frecuentes de indentación y organización, pero permite revisar la solución. 2 pts	No existe repositorio/hash verificable o el código no puede ejecutarse o revisarse. 0 pts

Puntaje obtenido: _____ / 60 puntos Nivel global referencial: ☐ Destacado ☐ Habilitado ☐ En desarrollo ☐ No logrado

Criterio de aplicación

El nivel se determina mediante el funcionamiento observable del commit entregado y las evidencias solicitadas. Las mejoras no solicitadas no compensan requisitos obligatorios ausentes.